

目录

- 1 实验目的
- 2 实验平台及环境
- 3 实验内容
- 4 实验步骤
- 5 实验结果及分析
 - 5.1 创建表
 - 5.1.1 分析
 - 5.1.2 执行过程及结果
 - 5.1.3 SQL 语句分析
 - 5.1.3.1 商品分类基本信息表
 - 5.1.3.2 用户基本信息表
 - 5.1.3.3 商品基本信息表
 - 5.1.3.4 地址基本信息表
 - 5.1.3.5 订单基本信息表
 - 5.1.3.6 订单明细基本信息表
 - 5.1.3.7 购物车基本信息表
 - 5.2 导入数据
 - 5.2.1 分析
 - 5.2.2 执行过程及结果
 - 5.3 操作数据表
- 6 实验小结

实验二 数据库表的创建与维护实验

1 实验目的

- 1. 通过进行数据库表的建立操作，熟悉并掌握 GaussDB 数据库表的建立方法，理解关系型数据库表的结构，巩固 PostgreSQL 中关于数据库表的建立语句；
- 2. 通过进行数据库表数据的增加、删除和插入等维护操作，熟悉并掌握 GaussDB 数据库数据的操作方法，巩固 PostgreSQL 中关于数据维护的语句；

2 实验平台及环境

配置项	参数
华为云	GaussDB v2.0-8.210.0
数据库	PostgreSQL

3 实验内容

建立相应的表并熟悉基本操作，例如建表、对表进行增、删、改、查。

4 实验步骤

 Note

该部分梳理了实验的基本步骤流程，详细的实验过程及相应结果统一放在下一部分。

- 1. 熟悉课程实验背景知识；
- 2. 使用 GaussDB 数据库软件创建相应的表；
- 3. 将提供的数据导入各表，掌握 GaussDB 数据库数据导入的方法；

 Warning

注意：

- 1. 表中空列的处理；
- 2. 表结构与数据类型的匹配。
- 3. 由于有约束关系，只能建一张数据库表导入一张表，不能一次性全部创建完后导入。

- 4. 修改“用户基本信息”表数据，增加名为“备注”的列，数据类型为 varchar() 型；
- 5. 修改“用户基本信息”表数据，将“备注”列的数据类型改为 int；
- 6. 修改“用户基本信息”表数据，删除“备注”列；
- 7. 删除“用户基本信息”数据表；

5 实验结果及分析

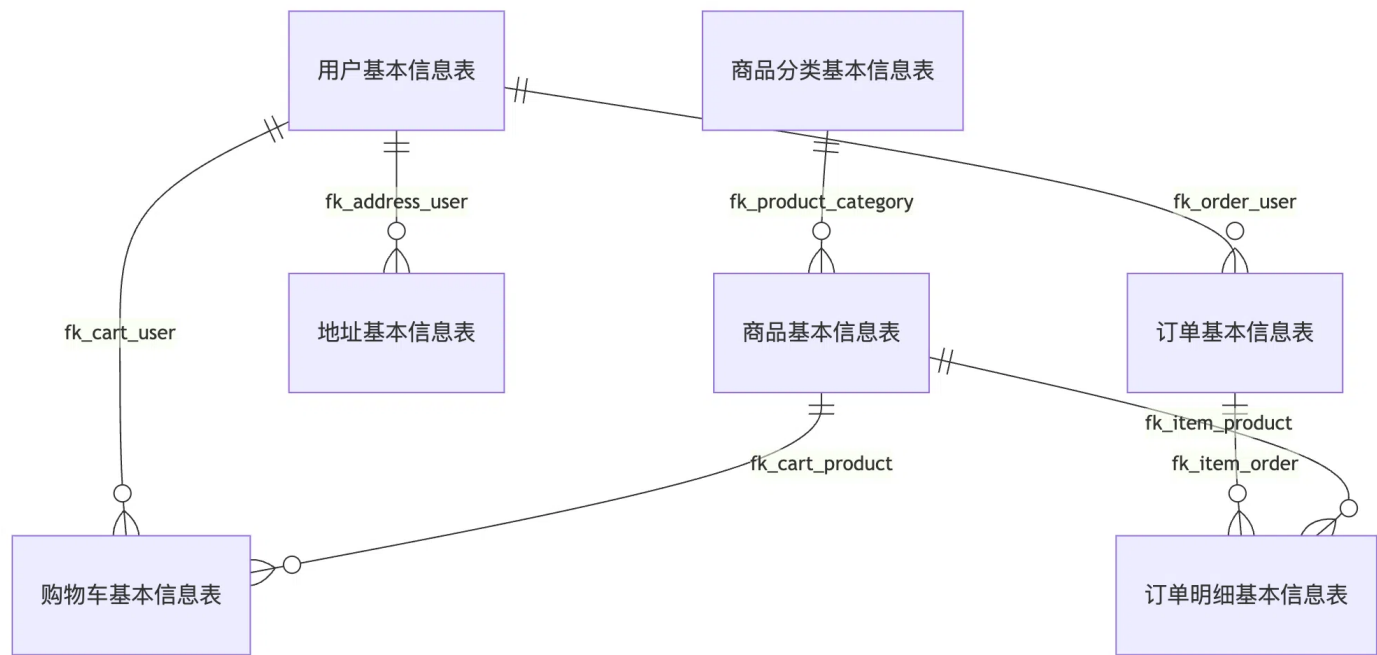
 Note

按照课上要求，实际创建的数据库命名为 `bupt2023212539` 而非“电商数据”，以便老师检查。

5.1 创建表

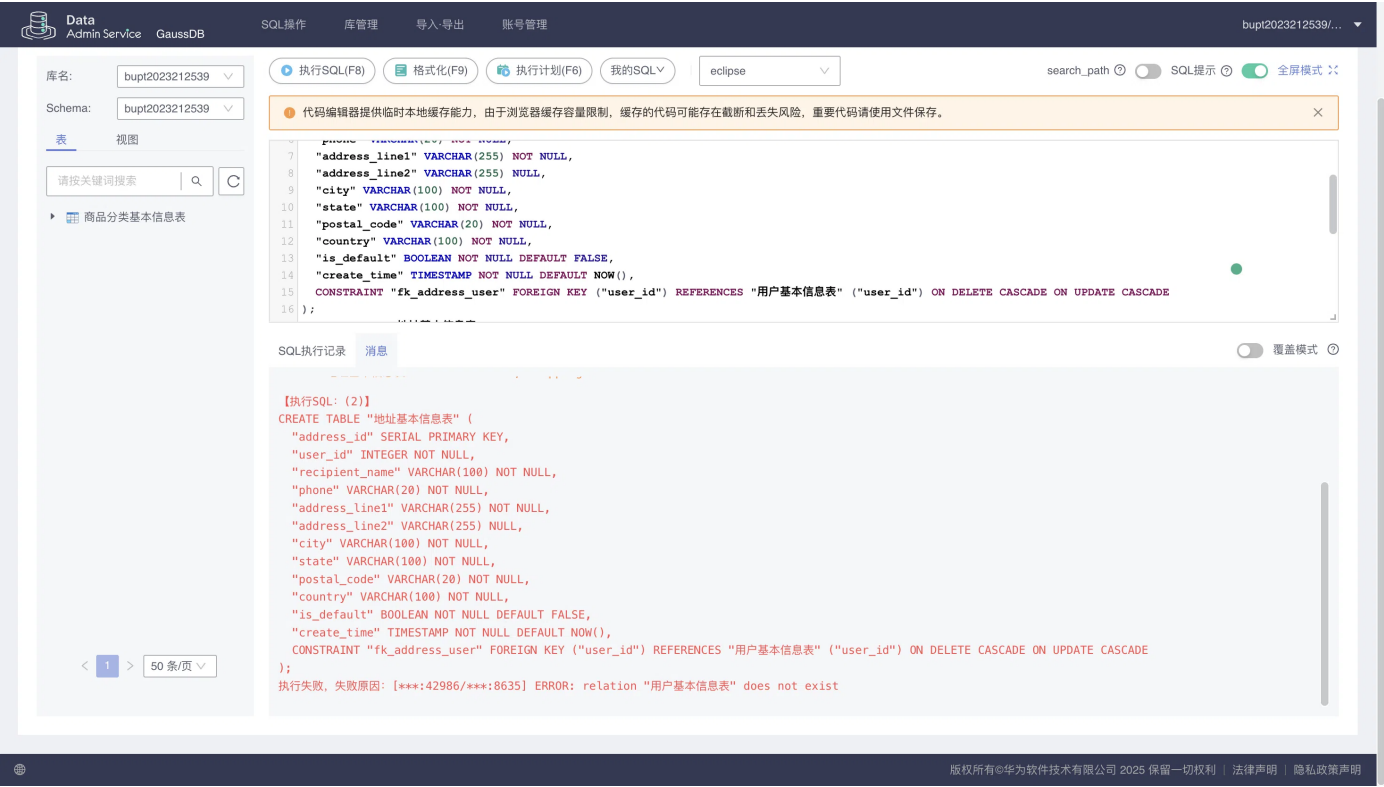
5.1.1 分析

在创建表之前，需要注意实验手册中的提示：*由于有约束关系，只能建一张数据库表导入一张表，不能一次性全部创建完后导入。*对待创建的表进行分析，可以发现这几张表之间有明显的外键约束关系，用 ER 图画出依赖顺序如下：



- “商品基本信息表” 依赖 “商品分类基本信息表”。
- “订单明细基本信息表” 同时依赖 “订单基本信息表” 和 “商品基本信息表”。
- “购物车基本信息表” 同时依赖 “用户基本信息表” 和 “商品基本信息表”。
- “地址基本信息表”、“订单基本信息表” 依赖 “用户基本信息表”。

为了验证，我不按依赖顺序，尝试先创建“地址基本信息表”，会发现报错如下：



理论上来说，一次性创建所有表再进入数据导入应该是可行的，不过前提是不勾选“清空表，执行导入前先对相应的表执行 Truncate 操作”的选项，否则会报错：

```
1 [2025-10-01 18:56:10] : 导入文件 商品分类基本信息表_1759315772804.csv 遇到异常: Failed to read the CSV file from the OBS bucket. ImexportException: Failed to prepare for writing the CSV file. com.huawei.opengauss.jdbc.util.PSQLException: [100.125.160.88:56548/100.112.133.2:8635] ERROR: cannot truncate a table referenced in a foreign key constraint Detail: Table "商品基本信息表" references "商品分类基本信息表". Hint: Truncate table "商品基本信息表" at the same time, or use TRUNCATE ... CASCADE.
```

这是因为 PostgreSQL 不允许直接清空被其他表引用的表，否则会破坏外键约束，即使实际数据并未导入。

经过尝试，我遵循实验手册要求“建一张数据库表导入一张表”进行。

5.1.2 执行过程及结果

1. 根据以上分析，按照依赖顺创建表，执行和运行结果如下图所示：

分析后发现是我们在建表时操作的自动提示：

- `SERIAL` 会自动创建序列；
- `PRIMARY KEY` 会自动创建唯一索引；

5.1.3 SQL 语句分析

我在下面分步展示创建表的 SQL 语句并给出对应分析：

5.1.3.1 商品分类基本信息表

```
1 DROP TABLE IF EXISTS "商品分类基本信息表";
2 CREATE TABLE "商品分类基本信息表" (
3     "category_id" SERIAL PRIMARY KEY,
4     "name" VARCHAR(100) NOT NULL,
5     "parent_id" INTEGER NULL,
6     "create_time" TIMESTAMP NOT NULL DEFAULT NOW()
7 );
```

其中针对 PostgreSQL 语句规则的分析如下（下文再出现类似语句时不再重复叙述）：

- `DROP TABLE IF EXISTS "{Table_Name}";`：表示如果数据库中已经存在名称相同的表，则先删除它，同时避免表不存在时报错；
- `CREATE TABLE`：新建表语句，后面括号中的内容为参数：
 - `"xxx" SERIAL PRIMARY KEY`：
 - `SERIAL` 表示自动增长的整数类型（PostgreSQL 会自动创建一个 sequence 序列）；
 - `PRIMARY KEY` 表示设为主键；
 - `"xxx" VARCHAR(100) NOT NULL`：
 - `VARCHAR(y)` 表示最大长度为 y 个字符；
 - `NOT NULL` 表示不能为空；
 - `"xxx" INTEGER NULL`：
 - `INTEGER` 表示整数类型；
 - `NULL` 表示允许为空；
 - `"create_time" TIMESTAMP NOT NULL DEFAULT NOW()`
 - `TIMESTAMP` 表示日期时间类型（精确到秒）；
 - `DEFAULT NOW()` 表示插入时如果没有指定 `create_time`，就自动填入当前时间；

上述语句的整体功能为：设计并创建了一张“商品分类基本信息表”，其中

- `category_id` 唯一标识分类（自动递增主键）。
- `name` 存放分类名称，必填。
- `parent_id` 允许存放上级分类，支持多级分类体系。
- `create_time` 记录创建时间，自动填充。

5.1.3.2 用户基本信息表

```

1 DROP TABLE IF EXISTS "用户基本信息表";
2 CREATE TABLE "用户基本信息表" (
3     "user_id" SERIAL PRIMARY KEY,
4     "username" VARCHAR(50) NOT NULL,
5     "password" VARCHAR(255) NOT NULL,
6     "email" VARCHAR(100) NOT NULL UNIQUE,
7     "phone" VARCHAR(20) NULL,
8     "age" INTEGER NULL,
9     "gender" VARCHAR(10) NULL,
10    "create_time" TIMESTAMP NOT NULL DEFAULT NOW(),
11    "update_time" TIMESTAMP NOT NULL DEFAULT NOW()
12 );

```

其中针对 PostgreSQL 语句规则的分析如下（下文再出现类似语句时不再重复叙述）：

- `"xxx" VARCHAR(100) NOT NULL UNIQUE` :
 - `UNIQUE` 表示要求唯一，不能重复；
- `"update_time" TIMESTAMP NOT NULL DEFAULT NOW()` :
 - `DEFAULT NOW()` 表示默认值为当前时间，用于记录更新时间（实际应用中通常结合 ON UPDATE 触发器或函数来自动更新）；

上述语句的整体功能为：设计并创建了一张“用户基本信息表”，其中

- **user_id**：唯一标识用户（自增主键）；
- **username**：用户名，必填；
- **password**：密码，必填；
- **email**：邮箱地址，必填且唯一；
- **phone**：手机号，可选；
- **age**：用户年龄，可选；
- **gender**：性别，可选；
- **create_time**：记录用户注册时间，自动填充；
- **update_time**：记录用户信息最后更新时间，默认当前时间；

5.1.3.3 商品基本信息表

```

1 DROP TABLE IF EXISTS "商品基本信息表";
2 CREATE TABLE "商品基本信息表" (
3     "product_id" SERIAL PRIMARY KEY,
4     "name" VARCHAR(255) NOT NULL,
5     "description" TEXT NULL,
6     "price" DECIMAL(10, 2) NOT NULL,
7     "stock" INTEGER NOT NULL DEFAULT 0,
8     "category_id" INTEGER NULL,
9     "create_time" TIMESTAMP NOT NULL DEFAULT NOW(),
10    "update_time" TIMESTAMP NOT NULL DEFAULT NOW(),
11    CONSTRAINT "fk_product_category" FOREIGN KEY ("category_id") REFERENCES "商品
    分类基本信息表" ("category_id") ON DELETE SET NULL ON UPDATE CASCADE
12 );

```

其中针对 PostgreSQL 语句规则的分析如下（下文再出现类似语句时不再重复叙述）：

- `"xxx" TEXT NULL`:
 - `TEXT` 表示长文本类型，没有长度限制；
- `"xxx" DECIMAL(10, 2) NOT NULL`
 - `DECIMAL(10, 2)` 表示定点小数，总共 10 位，其中 2 位小数；
- `CONSTRAINT "fk_product_category" FOREIGN KEY ("category_id") REFERENCES "商品分类基本信息表" ("category_id") ON DELETE SET NULL ON UPDATE CASCADE`
 - `CONSTRAINT "fk_product_category"`: 定义外键约束并命名为 `fk_product_category`;
 - `FOREIGN KEY ("category_id")`: 表示本表的 `category_id` 字段是外键;
 - `REFERENCES "商品分类基本信息表" ("category_id")`: 该外键引用 `商品分类基本信息表` 的 `category_id`;
 - `ON DELETE SET NULL`: 当分类被删除时，商品的 `category_id` 自动设为 `NULL`（保留商品，但去掉分类）;
 - `ON UPDATE CASCADE`: 当分类的 `category_id` 被更新时，商品表里的 `category_id` 自动同步更新;

上述语句的整体功能为：创建了“商品基本信息表”，主要功能是存储商品的基本信息，并通过外键与“商品分类基本信息表”建立关联。

5.1.3.4 地址基本信息表

```

1 DROP TABLE IF EXISTS "地址基本信息表";
2 CREATE TABLE "地址基本信息表" (
3     "address_id" SERIAL PRIMARY KEY,
4     "user_id" INTEGER NOT NULL,
5     "recipient_name" VARCHAR(100) NOT NULL,
6     "phone" VARCHAR(20) NOT NULL,
7     "address_line1" VARCHAR(255) NOT NULL,
8     "address_line2" VARCHAR(255) NULL,
9     "city" VARCHAR(100) NOT NULL,
10    "state" VARCHAR(100) NOT NULL,
11    "postal_code" VARCHAR(20) NOT NULL,
12    "country" VARCHAR(100) NOT NULL,
13    "is_default" BOOLEAN NOT NULL DEFAULT FALSE,
14    "create_time" TIMESTAMP NOT NULL DEFAULT NOW(),
15    CONSTRAINT "fk_address_user" FOREIGN KEY ("user_id") REFERENCES "用户基本信息表"
16    ("user_id") ON DELETE CASCADE ON UPDATE CASCADE
17 );

```

其中针对 PostgreSQL 语句规则的分析如下（下文再出现类似语句时不再重复叙述）：

- `"xxx" BOOLEAN NOT NULL DEFAULT FALSE`:
 - `BOOLEAN` 表示布尔型变量；
- `CONSTRAINT "fk_address_user" FOREIGN KEY ("user_id") REFERENCES "用户基本信息表" ("user_id") ON DELETE CASCADE ON UPDATE CASCADE`:
 - `CONSTRAINT "fk_address_user"`: 定义外键约束并命名为 `fk_address_user`;
 - `FOREIGN KEY ("user_id")`: 表示本表的 `user_id` 字段是外键;
 - `REFERENCES "用户基本信息表" ("user_id")`: 该外键引用 `用户基本信息表` 的 `user_id`;
 - `ON DELETE CASCADE`: 当用户被删除时，地址表中对应的地址记录也会被自动删除;

- `ON UPDATE CASCADE`：当用户的 `user_id` 被更新时，地址表中的 `user_id` 会自动同步更新；

上述语句的整体功能为：创建了“地址基本信息表”，主要功能是存储用户的收货地址信息，并通过外键与“用户基本信息表”建立关联，实现用户与地址的对应关系，同时支持用户删除或更新时地址记录的级联操作。

5.1.3.5 订单基本信息表

```
1 DROP TABLE IF EXISTS "订单基本信息表";
2 CREATE TABLE "订单基本信息表" (
3     "order_id" SERIAL PRIMARY KEY,
4     "user_id" INTEGER NOT NULL,
5     "total_amount" DECIMAL(10, 2) NOT NULL,
6     "create_time" TIMESTAMP NOT NULL DEFAULT NOW(),
7     "update_time" TIMESTAMP NOT NULL DEFAULT NOW(),
8     CONSTRAINT "fk_order_user" FOREIGN KEY ("user_id") REFERENCES "用户基本信息表"
9     ("user_id") ON DELETE CASCADE ON UPDATE CASCADE
10 );
```

其中针对 PostgreSQL 语句规则的分析如下（下文再出现类似语句时不再重复叙述）：

- `CONSTRAINT "fk_order_user" FOREIGN KEY ("user_id") REFERENCES "用户基本信息表" ("user_id") ON DELETE CASCADE ON UPDATE CASCADE`：
 - `CONSTRAINT "fk_order_user"`：定义外键约束并命名为 `fk_order_user`；
 - `FOREIGN KEY ("user_id")`：表示本表的 `user_id` 字段是外键；
 - `REFERENCES "用户基本信息表" ("user_id")`：该外键引用 `用户基本信息表` 的 `user_id`；
 - `ON DELETE CASCADE`：当用户被删除时，订单表中对应的订单记录也会被自动删除；
 - `ON UPDATE CASCADE`：当用户的 `user_id` 被更新时，订单表中的 `user_id` 会自动同步更新；

上述语句的整体功能为：创建了“订单基本信息表”，主要功能是存储用户的订单信息，并通过外键与“用户基本信息表”建立关联，实现用户与订单的对应关系，同时支持用户删除或更新时订单记录的级联操作。

5.1.3.6 订单明细基本信息表

```
1 DROP TABLE IF EXISTS "订单明细基本信息表";
2 CREATE TABLE "订单明细基本信息表" (
3     "order_item_id" SERIAL PRIMARY KEY,
4     "order_id" BIGINT NOT NULL,
5     "product_id" INTEGER NOT NULL,
6     "quantity" INTEGER NOT NULL,
7     "price" DECIMAL(10, 2) NOT NULL,
8     "create_time" TIMESTAMP NOT NULL DEFAULT NOW(),
9     CONSTRAINT "fk_item_order" FOREIGN KEY ("order_id") REFERENCES "订单基本信息表"
10     ("order_id") ON DELETE CASCADE ON UPDATE CASCADE,
11     CONSTRAINT "fk_item_product" FOREIGN KEY ("product_id") REFERENCES "商品基本信息表"
12     ("product_id") ON DELETE RESTRICT ON UPDATE CASCADE
13 );
```

其中针对 PostgreSQL 语句规则的分析如下（下文再出现类似语句时不再重复叙述）：

- `CONSTRAINT "fk_item_order" FOREIGN KEY ("order_id") REFERENCES "订单基本信息表" ("order_id") ON DELETE CASCADE ON UPDATE CASCADE`：

- `CONSTRAINT "fk_item_order"`: 定义外键约束并命名为 `fk_item_order`;
- `FOREIGN KEY ("order_id")`: 表示本表的 `order_id` 字段是外键;
- `REFERENCES "订单基本信息表" ("order_id")`: 该外键引用 **订单基本信息表** 的 `order_id`;
- `ON DELETE CASCADE`: 当订单被删除时, 订单明细表中对应的明细记录也会被自动删除;
- `ON UPDATE CASCADE`: 当订单的 `order_id` 被更新时, 订单明细表中的 `order_id` 会自动同步更新;
- `CONSTRAINT "fk_item_product" FOREIGN KEY ("product_id") REFERENCES "商品基本信息表" ("product_id") ON DELETE RESTRICT ON UPDATE CASCADE`:
 - `CONSTRAINT "fk_item_product"`: 定义外键约束并命名为 `fk_item_product`;
 - `FOREIGN KEY ("product_id")`: 表示本表的 `product_id` 字段是外键;
 - `REFERENCES "商品基本信息表" ("product_id")`: 该外键引用 **商品基本信息表** 的 `product_id`;
 - `ON DELETE RESTRICT`: 不允许删除商品表中有对应订单明细的商品 (防止删除已有订单的商品);
 - `ON UPDATE CASCADE`: 当商品的 `product_id` 被更新时, 订单明细表中的 `product_id` 会自动同步更新;

上述语句的整体功能为: 创建了“订单明细基本信息表”, 主要功能是存储每个订单的明细信息, 并通过外键与“订单基本信息表”和“商品基本信息表”建立关联, 实现订单与明细、明细与商品的对应关系, 同时支持订单删除时明细自动删除, 而商品删除时受限制以保证数据一致性。

5.1.3.7 购物车基本信息表

```

1 DROP TABLE IF EXISTS "购物车基本信息表";
2 CREATE TABLE "购物车基本信息表" (
3     "cart_id" SERIAL PRIMARY KEY,
4     "user_id" INTEGER NOT NULL,
5     "product_id" INTEGER NOT NULL,
6     "quantity" INTEGER NOT NULL,
7     "create_time" TIMESTAMP NOT NULL DEFAULT NOW(),
8     CONSTRAINT "fk_cart_user" FOREIGN KEY ("user_id") REFERENCES "用户基本信息表"
9     ("user_id") ON DELETE CASCADE ON UPDATE CASCADE,
10    CONSTRAINT "fk_cart_product" FOREIGN KEY ("product_id") REFERENCES "商品基本信
    息表" ("product_id") ON DELETE CASCADE ON UPDATE CASCADE
11 );

```

- `CONSTRAINT "fk_cart_user" FOREIGN KEY ("user_id") REFERENCES "用户基本信息表" ("user_id") ON DELETE CASCADE ON UPDATE CASCADE`:
 - `CONSTRAINT "fk_cart_user"`: 定义外键约束并命名为 `fk_cart_user`;
 - `FOREIGN KEY ("user_id")`: 表示本表的 `user_id` 字段是外键;
 - `REFERENCES "用户基本信息表" ("user_id")`: 该外键引用 **用户基本信息表** 的 `user_id`;
 - `ON DELETE CASCADE`: 当用户被删除时, 购物车中对应的记录也会被自动删除;
 - `ON UPDATE CASCADE`: 当用户的 `user_id` 被更新时, 购物车表中的 `user_id` 会自动同步更新;
- `CONSTRAINT "fk_cart_product" FOREIGN KEY ("product_id") REFERENCES "商品基本信息表" ("product_id") ON DELETE CASCADE ON UPDATE CASCADE`:
 - `CONSTRAINT "fk_cart_product"`: 定义外键约束并命名为 `fk_cart_product`;
 - `FOREIGN KEY ("product_id")`: 表示本表的 `product_id` 字段是外键;
 - `REFERENCES "商品基本信息表" ("product_id")`: 该外键引用 **商品基本信息表** 的 `product_id`;
 - `ON DELETE CASCADE`: 当商品被删除时, 购物车中对应的记录也会被自动删除;

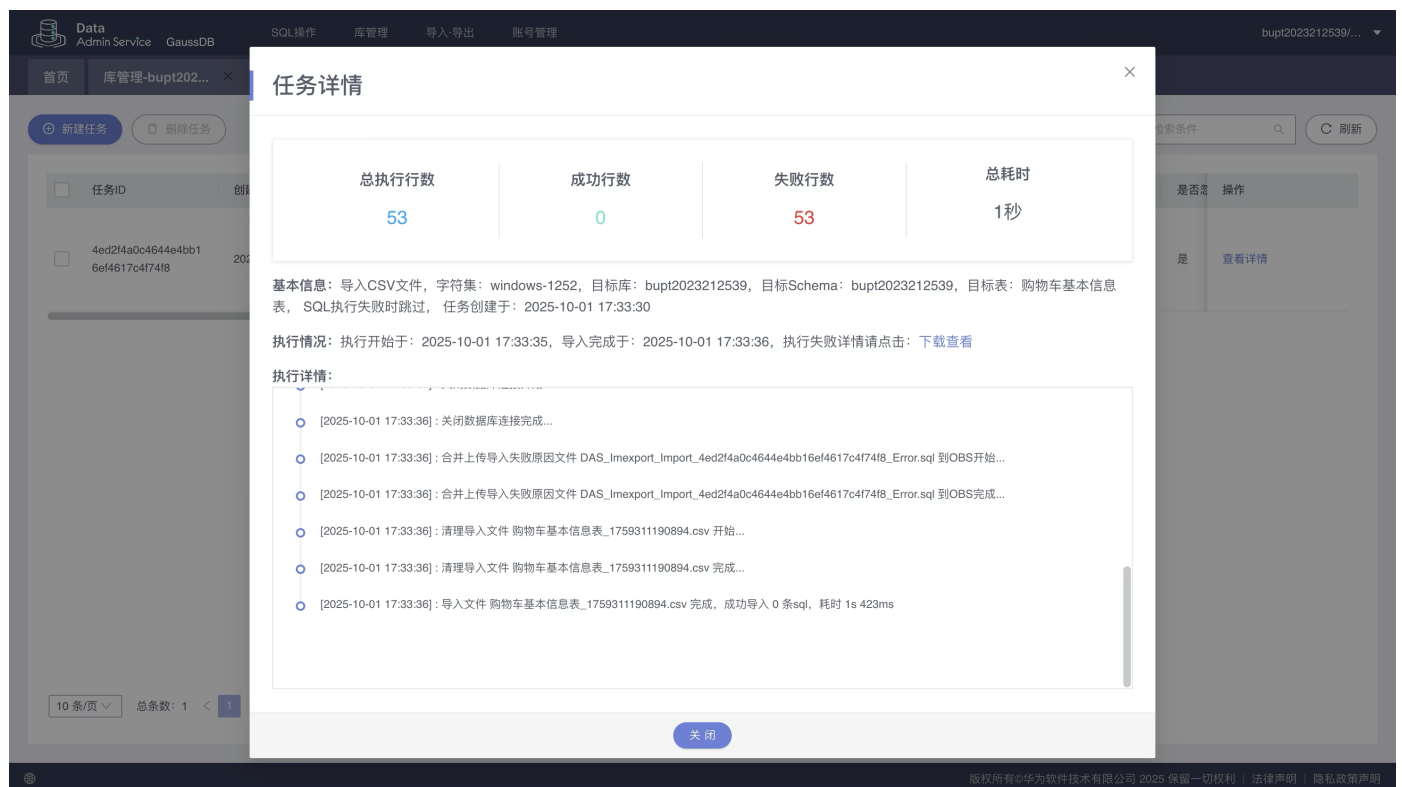
- `ON UPDATE CASCADE`：当商品的 `product_id` 被更新时，购物车表中的 `product_id` 会自动同步更新；

上述语句的整体功能为：创建了“购物车基本信息表”，主要功能是存储用户的购物车信息，并通过外键与“用户基本信息表”和“商品基本信息表”建立关联，实现用户与购物车、商品与购物车的对应关系，同时支持用户或商品删除时购物车记录自动删除，保证数据一致性。

5.2 导入数据

5.2.1 分析

同样，再导入数据表时，同样需要遵循依赖关系，如果父表中的数据未导入，先导入子表中的数据，会导入失败：



具体的报错原因如下：

```
1  =====
2  [error sql]:
3  insert into "bupt2023212539"."?????????"
   ("cart_id","user_id","product_id","quantity","create_time") values
   ('1','42','15','3','2021/7/18')
4  [error sql line number]:
5  2
6  [error message]:
7  [100.125.166.40:33504/100.112.133.2:8635] ERROR: insert or update on table
   "?????????" violates foreign key constraint "fk_cart_user"
8  Detail: Key (user_id)=(42) is not present in table "?????????".
```

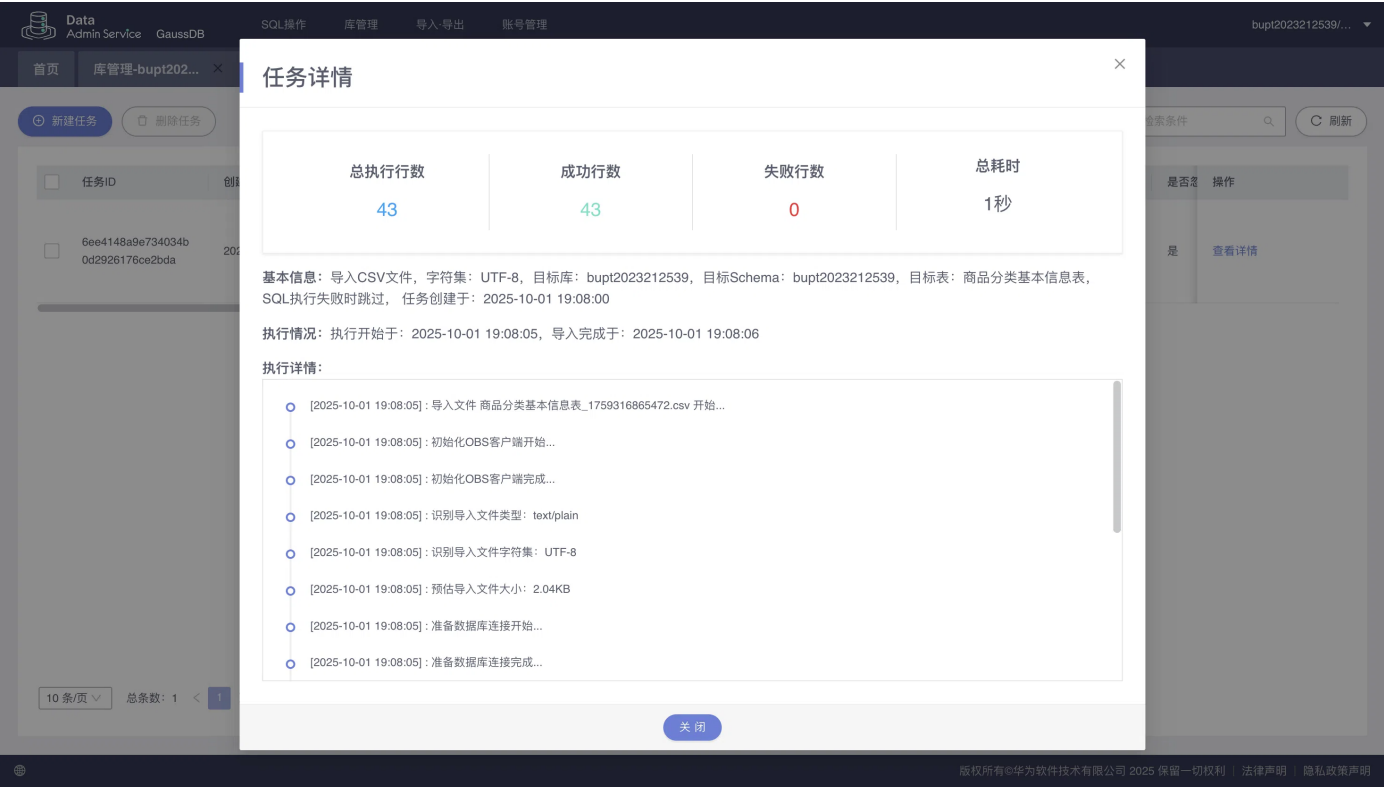
即“外键约束失败”，这是因为 PostgreSQL 是强一致性数据库，默认在插入时会立即检查外键。

5.2.2 执行过程及结果

1. 点击“SQL窗口”，导入配置如下：



2. 创建并等待执行结果：



3. 依次创建表并导入其他数据，此处省略演示，最终结果如下，经确认所有行均成功导入；

Admin Service GaussDB SQL操作 库管理 导入·导出 账号管理 bupt2023212539/...

首页 库管理-bupt202... SQL窗口

当前所在库: **bupt2023212539** | 实例名称: gauss-391d | 192.168.0.53:8000 | 字符集: UTF8 | 时区: Etc/GMT-8 | SQL保存到执行记录

库名: bupt2023212539 Schema: bupt2023212539

表 视图

请按关键词搜索

商品分类基本信息表
商品基本信息表
地址基本信息表
用户基本信息表

columns

user_id (int4(32,0))
username (varchar(50))
password (varchar(255))
email (varchar(100))
phone (varchar(20))
age (int4(32,0))
gender (varchar(10))
create_time (timestamp)
update_time (timestamp)
备注 (varchar(255))

indexes

执行SQL(F8) 格式化(F9) 执行计划(F6) 我的SQL eclipse

代码编辑器提供临时本地缓存能力, 由于浏览器缓存容量限制, 缓存的代码可能存在截断和丢失风险, 重要代码请使用文件保存。

```
1 ALTER TABLE "用户基本信息表"  
2 ADD COLUMN "备注" VARCHAR(255);  
3
```

SQL执行记录 消息

执行时间	SQL语句	消耗时间	执行结果
2025-10-01 19:35:53	ALTER TABLE "用户基本信息表" ADD COLUMN "备注" VARCHAR(255);	4 ms	执行成功
2025-10-01 19:35:52	set time zone 'Etc/GMT-8'	3 ms	执行成功
2025-10-01 19:32:14	select * from "bupt2023212539"."用户基本信息表"	6 ms	执行成功
2025-10-01 18:57:23	select * from "bupt2023212539"."商品基本信息表"	5 ms	执行成功
2025-10-01 16:23:23	CREATE TABLE "购物车基本信息表" ("cart_id" SERIAL PRIMARY KEY, "user_id" INTEGER NOT NULL, "product_id" INTEGER NOT NULL, "quantity" INTEGER NOT NULL, "create_time" TIMESTAMP NOT NULL DEFAULT NOW(), CONSTRAINT "fk_cart_user" FOREIGN KEY ("user_id") REFERENCES "用户基本信息表" ("user_id") ON DELETE CASCADE ON UPDATE CASCADE, CONSTRAINT "fk_cart_product" FOREIGN KEY ("product_id") REFERENCES "商品基本信息表" ("product_id") ON DELETE CASCADE ON UPDATE CASCADE)	12 ms	执行成功

版权所有©华为软件技术有限公司 2025 保留一切权利 | 法律声明 | 隐私政策声明

2. 修改“用户基本信息”表数据, 将“备注”列的数据类型改为int;

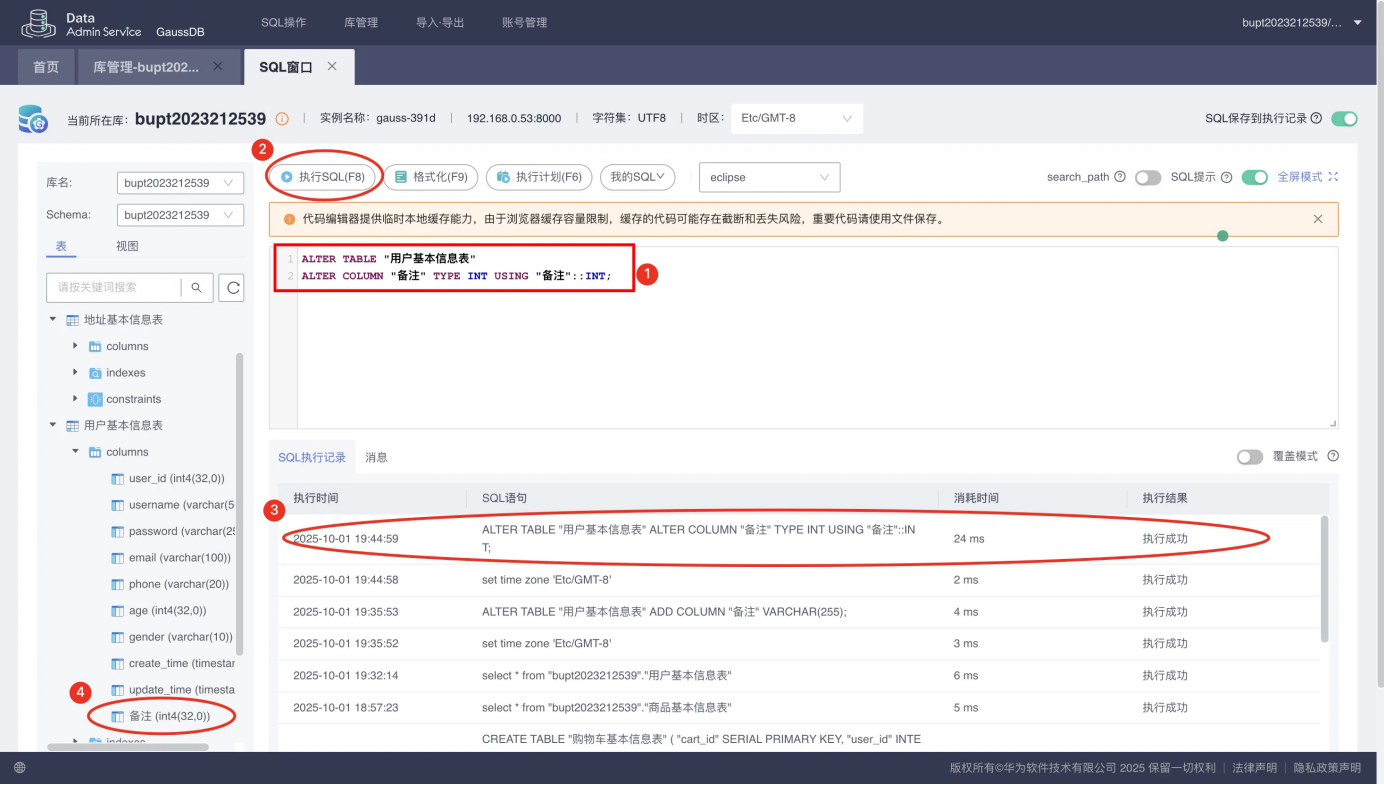
该操作对应的 PostgreSQL 语句为:

```
1 ALTER TABLE "用户基本信息表"  
2 ALTER COLUMN "备注" TYPE INT USING "备注"::INT;
```

其中:

- `ALTER TABLE "用户基本信息表"` 表示指定要修改的表。
- `ALTER COLUMN "备注" TYPE INT` 表示将备注列的数据类型修改为 INT;
- `USING "备注"::INT` 表示如果原来的备注列是字符串, 需要显式转换为整数;
 - 如果新旧类型可以自动转换 (比如 `INT` → `BIGINT`), 可以省略 `USING`;
 - 如果列中有 'abc' 这样的非数字字符串, 执行会失败; (由于现在备注列均为 NULL, 所有可以进行类型转换);

我在 SQL 窗口中执行, 结果如下:



3. 修改“用户基本信息”表数据，删除“备注”列；

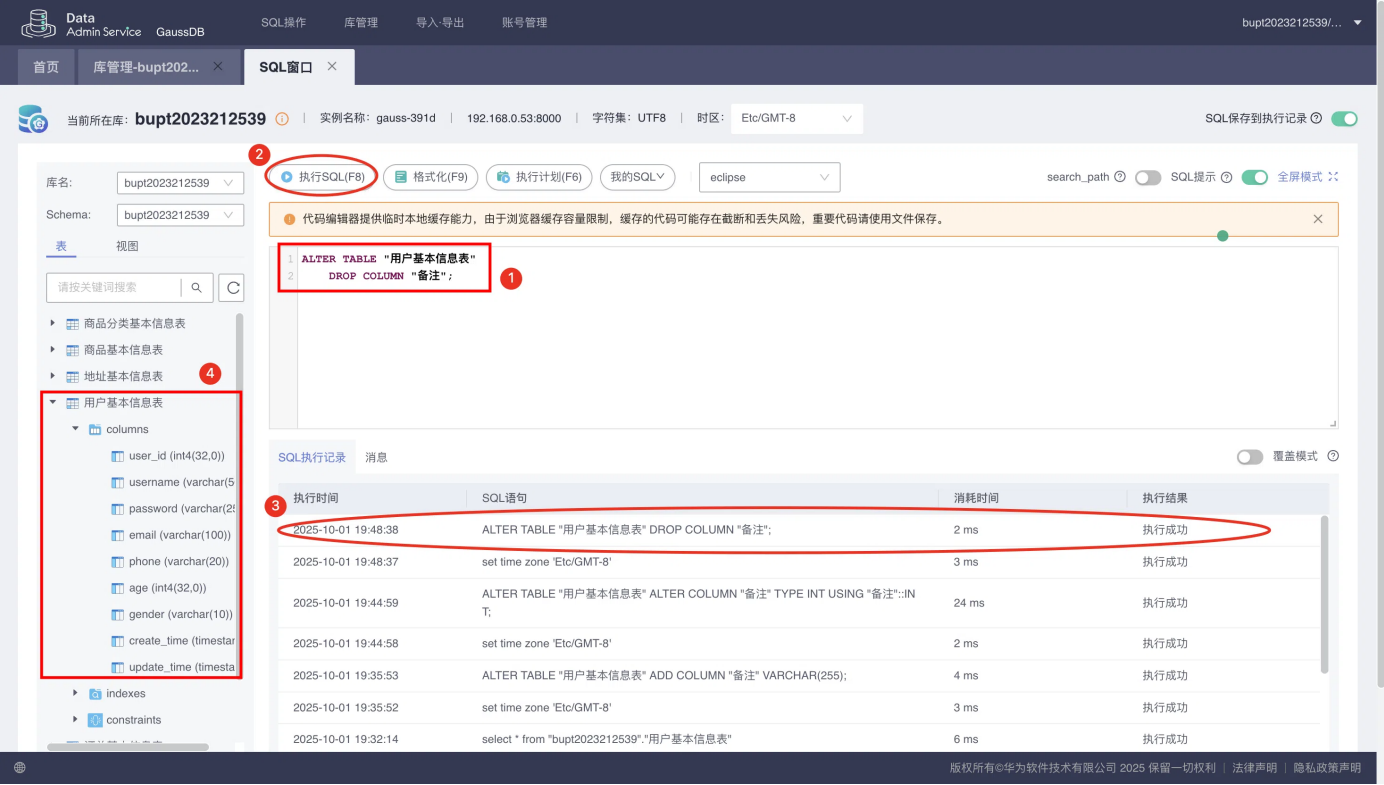
该操作对应的 PostgreSQL 语句为：

```
1 ALTER TABLE "用户基本信息表"
2 DROP COLUMN "备注";
```

其中：

- ALTER TABLE "用户基本信息表" 表示指定要修改的表；
- DROP COLUMN "备注" 表示删除表中的备注列及其所有数据；

我在 SQL 窗口中执行，结果如下：



4. 删除“用户基本信息”数据表；

该操作对应的 PostgreSQL 语句为：

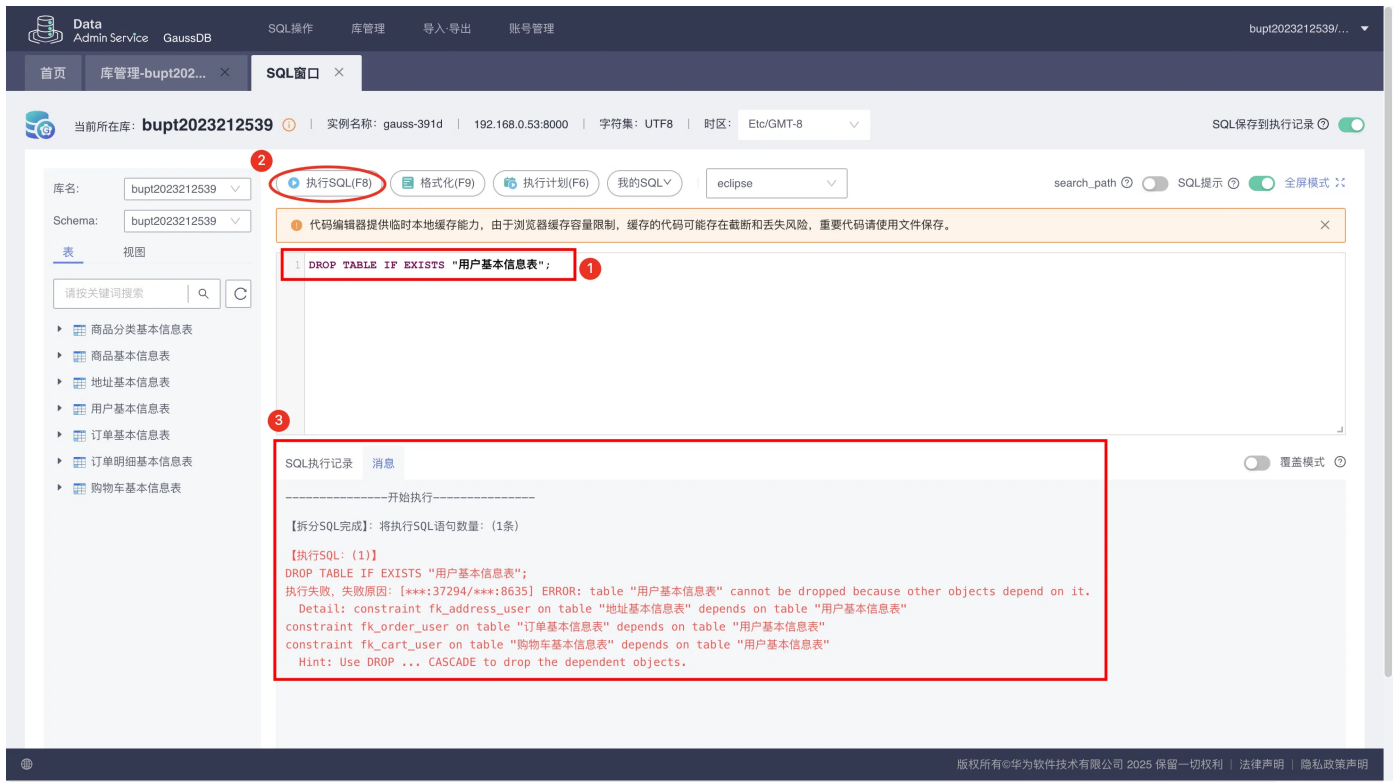
```
1 DROP TABLE IF EXISTS "用户基本信息表";
```

其中：

- DROP TABLE "用户基本信息表" 表示删除名为 "用户基本信息表" 的表；
- IF EXISTS 表示如果表不存在，则不会报错，保证安全执行；

需要注意的是，删除表会永久丢失该表的所有数据、索引、约束和触发器。同时如果其他表有外键引用“用户基本信息表”，直接删除会报错。

本实验中，“购物车基本信息表”和“地址基本信息表”均有外键引用“用户基本信息表”，实际执行结果如下：



如上图所示，报错原因与我的理论分析一致。如需删除该表，可以使用 CASCADE 级联删除引用该表的相关对象：

```
1 DROP TABLE IF EXISTS "用户基本信息表" CASCADE;
```

执行后会删除“用户基本信息表”、“购物车基本信息表”和“地址基本信息表”，由于后续实验需要，此处不再执行和演示删除效果。

6 实验小结

通过本次实验，我进一步熟悉了 GaussDB 中数据库和数据导入的有关操作，以及数据库表数据的增加、删除和插入等维护操作的 SQL 语句使用。

我遇到的一个问题是数据导入问题，一开始经过理论分析，我正确通过 ER 图分析了数据表之间的外键约束关系，认为只要遵循正确的顺序，可以一次性创建表并导入数据。不过实际尝试过程中发现，如果在导入时选择先清空数据表内容，就会报错外键约束，即使外键所在的表并无关联的数据。

通过理论分析 + 实践操作，让我清晰地理解了外键依赖关系，进一步加深了我对外键约束的理解，培养了我严谨使用数据库的工程能力。

一些建议：在课堂使用时，由于并发使用的客户端较多，华为云实例进行了限制，让我看到了华为对云数据库安全的重视，但我认为针对教学场景，可以适当开放充足且合理的资源供学生自由探索、学习和使用。